Job Agent — Project Context for Claude Code

What we're building

A personal AI job search agent. Scrapes Greenhouse, Lever, and HN Who's Hiring, scores every job against a user profile using an LLM, and surfaces the best matches. Two users (Manav + sister), fully isolated profiles, deployed to Railway.

Core philosophy

- Get signal fast — scraper + AI filter is the highest-value piece
- Incremental — each step is independently runnable
- Simple infra — SQLite now, Postgres before Railway deploy
- No overengineering — if it works for two people, it's done
- Learning — technologies chosen partly for career relevance (embeddings, LangGraph, MLflow directly applicable to agentic AI engineering roles)

Current status — what's actually in the repo

PHASE 1 — Prototype

Step	Status	Description
1	✅	Skeleton, config.yaml, db.py
2	✅	Greenhouse scraper
3	✅	HN Who's Hiring scraper
4	✅	LLM scorer — multi-dimension, ATS score, multi-provider, RateLimiter
4.5	✅	TheirStack company discovery + Lever scraper
5	✅	main.py orchestrator with full CLI flags
6	✅	Profile system + Streamlit onboarding (onboarding.py)
7	✅	Streamlit dashboard — 4 tabs, sidebar pipeline runner (dashboard.py)
8	✅	Pydantic models (models.py) + instructor integration in scorer.py

PHASE 2 — Harden + Deploy

Step	Status	Description
9	⬜	Loguru + pipeline run logging — scrape_runs table, structured logs
10	⬜	SQLite → PostgreSQL — multi-user write safety
11	⬜	Deploy to Railway

PHASE 3 — Smarter Agent

Step	Status	Description
12	⬜	Embeddings + ChromaDB — semantic job matching
13	⬜	MLflow experiment tracking
14	⬜	Feedback loop — thumbs up/down → labeled dataset
15	⬜	APScheduler — automatic runs

PHASE 4 — Agentic

Step	Status	Description
16	⬜	LangGraph — stateful decision graph
17	⬜	pgvector + FAISS
18	⬜	Fine-tuned classifier on feedback data
19	⬜	Celery + Redis

Actual file structure (as of latest commit)

```
job-agent/
├── config.yaml              # root config — CLI fallback when no --profile
├── db.py                    # SQLite, profile-aware, all DB logic
├── config.py                # load_config() with PyPDF2 resume extraction
```

```
├── scraper.py               # Greenhouse + Lever + HN scrapers
├── scorer.py                # LLM pipeline, RateLimiter, instructor integration
├── candidate_profile.py     # build_structured_profile(), confirm_profile()
├── text_utils.py            # extract_job_context() smart truncation
├── theirstack.py            # company discovery, slug resolution GH + Lever
├── models.py                # Pydantic: ScoreResult, StructuredProfile,
ScoringWeights
├── main.py                  # CLI orchestrator
├── onboarding.py            # Streamlit 5-step profile creation flow
├── dashboard.py             # Streamlit web UI entry point
├── test_step8.py            # Pydantic model tests
├── pytest.ini
├── requirements.txt
├── .env
├── .env.example
├── .gitignore
├── .claude/settings.local.json
├── profiles/
│   └── default/
│       └── config.yaml      # default profile with company lists
└── jobs.db                  # root DB (CLI use only)
```

Key module details

models.py (Step 8 — complete)

Three Pydantic v2 models:

ScoreResult — LLM output for job scoring

- Fields: `disqualified`, `disqualify_reason`, `role_fit`, `stack_match`, `seniority`, `location`, `growth`, `compensation` (all 0-10), `reasons` (list, max 4), `flags` (list), `one_liner`
- Validators: clamps scores to 0-10, zeros all dimensions when disqualified, limits reasons to 4 items, filters empty strings

StructuredProfile — LLM output for profile extraction

- Fields: `name`, `yoe`, `current_title`, `core_skills`, `languages`, `frameworks`, `cloud`, `past_roles`, `education`, `strengths`, `target_roles`, `min_salary`, `remote_preference`, `preferred_locations`

- Validators: deduplicates all list fields, parses salary from string ($130,000), clamps yoe to >= 0

ScoringWeights — validation only, loaded from config

- Fields: all 6 dimension weights as floats
- Validator: weights must sum to 1.0 ± 0.01

scorer.py

- `get_llm_client(config) -> LlmCall` — returns `(prompt, max_tokens) -> (str, int)`
- `get_instructor_client(config)` — returns instructor-wrapped client for structured output
- `RateLimiter(max_rpm, max_tpm, max_rpd)` — rolling 60s window, hard RPD stop via sys.exit(0)
- `keyword_prescore(job, config) -> float` — 0.0–1.0, skip LLM if < 0.15
- `score_dimensions(job, config, llm_call, structured_profile) -> dict` — single LLM call, returns disqualified + all dimension scores + reasons/flags/one_liner. Adjusts prompt for internship vs fulltime based on `config["profile"]["job_type"]`.
- `compute_ats_score(job, config) -> dict` — pure Python, keyword overlap
- `score_all_jobs(config, yes=False, profile=None) -> list` — full pipeline
- `print_results(results, config)` — terminal display

db.py

Profile resolution order: explicit `profile=` arg → `_ACTIVE_PROFILE` module var → root jobs.db

Key functions (all accept `profile=None`):

- `set_active_profile(profile)` / `get_db_path(profile=None)`
- `init_db(profile=None)` — creates profile folder if needed, runs migrations
- `insert_job(job, profile=None) -> bool`
- `save_score(job_id, ..., profile=None)`
- `get_unscored(profile=None) -> list[Job]` — jobs with score IS NULL and score_attempts < 3
- `get_top_jobs(min_score, profile=None) -> list`
- `count_jobs(profile=None) -> dict` — {total, scored}
- `rescore_reset(profile=None)` — clears scores, resets score_attempts to 0

- `update_job_status(job_id, status, profile=None)`
- `save_discovered_slug(slug, company_name, ats, profile=None)`
- `load_discovered_slugs(ats, profile=None) -> list[str]`
- `increment_score_attempts(job_id, profile=None)`
- `write_score_error(job_id, error, profile=None)`

Job dataclass

```python
@dataclass
class Job:
    id: str           # "{source}_{source_id}"
    title: str
    company: str
    location: str
    url: str
    raw_text: str     # cleaned, capped at 2000 chars
    source: str       # "greenhouse" | "lever" | "hackernews"
    score_attempts: int = 0
    score_error: Optional[str] = None
    status: str = "new"
```

Scores table

```sql
```

```sql
CREATE TABLE scores (
    job_id       TEXT PRIMARY KEY,
    fit_score    INTEGER,
    role_fit     INTEGER,
    stack_match  INTEGER,
    seniority    INTEGER,
    location     INTEGER,
    growth       INTEGER,
    compensation INTEGER,
    reasons      TEXT,       -- JSON array, max 4 items
    flags        TEXT,       -- JSON array
    skill_misses TEXT,       -- JSON array, top 5 missing skills
    one_liner    TEXT,
    ats_score    INTEGER,
    scored_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
```

config.yaml structure

```yaml
```

```yaml
profile:
  name: "..."
  job_type: "fulltime" | "internship"
  resume_file: "path/to/resume.pdf"   # OR use resume: inline text
  resume: |
    ...
  bio: |
    ...
  # internship-only:
  target_season: "Summer 2026"
  school: "..."
  major: "..."
  gpa: "..."
  graduation_year: 2027

llm:
  provider: "groq"
  model:
    groq: "meta-llama/llama-4-scout-17b-16e-instruct"
    groq_balanced: "llama-3.3-70b-versatile"
    groq_testing: "llama-3.1-8b-instant"
    gemini: "gemini-2.5-flash"
    gemini_lite: "gemini-2.5-flash-lite-preview-06-17"
    anthropic: "claude-sonnet-4-20250514"
    openai: "gpt-4o-mini"
  temperature: 0
  rate_limits:
    groq:      {max_rpm: 28, max_tpm: 28000, max_rpd: 950}
    gemini:    {max_rpm: 9,  max_tpm: 240000, max_rpd: 250}
    gemini_lite: {max_rpm: 14, max_tpm: 240000, max_rpd: 1000}
    anthropic: {max_rpm: 50, max_tpm: 9000000}
    openai:    {max_rpm: 50, max_tpm: 9000000}

preferences:
  titles: [...]
  desired_skills: [...]
  hard_no_keywords: [...]
  location:
    remote_ok: true
    preferred_locations: [...]
  compensation:
    min_salary: 130000
  yoe: 2
```

```yaml
  filters:
    min_yoe: 0
    max_yoe: 5
    max_job_age_days: 30
    require_degree_filter: true
    title_blocklist: [Staff, Principal, VP, Director, Head of, Manager, Lead]
    countries_allowed: [United States, US, USA, Remote]

sources:
  greenhouse:
    enabled: true
    companies: [anthropic, stripe, figma, databricks, ...]
  lever:
    enabled: true
    companies: [stripe, linear, vercel, notion, ...]
  hn:
    enabled: true   # note: key is "hn" not "hackernews" in sources

scoring:
  min_display_score: 60
  weights:
    role_fit:     0.30
    stack_match:  0.25
    seniority:    0.20
    location:     0.10
    growth:       0.10
    compensation: 0.05
```

scraper.py

All scrapers accept `profile=None` and call `init_db(profile=profile)` internally.

- `scrape_greenhouse(config, slugs, profile=None) -> dict` — {companies_checked, new_jobs_saved, errors}
- `scrape_lever(config, slugs, profile=None) -> dict`
- `scrape_hn(config, profile=None) -> dict` — {thread_found, new_jobs_saved, errors}
- `passes_filters(text, title, location, config, source, debug=False) -> bool`
- `strip_html(html_text) -> str` — handles double-encoded entities via html.unescape()

theirstack.py

- `fetch_companies(config) -> list` — TheirStack API, filtered to US tech 10-5000

employees

- `resolve_greenhouse_slug(company) -> str | None`
- `resolve_lever_slug(company) -> str | None`
- `get_or_discover_slugs(config, profile=None) -> dict[str, list[str]]` Returns {"greenhouse": [...], "lever": [...]}

onboarding.py

5-step Streamlit flow, all state in `st.session_state.onboarding_data`. Entry point: `render_onboarding()` — called by dashboard.py.

Steps: job_type → basic_info → preferences → llm_provider → review_create

Key functions:

- `generate_config(data) -> dict` — builds complete config.yaml from onboarding data
- `create_profile(data)` — writes config.yaml, copies PDF, calls init_db(profile=slug), upserts API key into root .env via `_upsert_env_key()`
- `sanitize_slug(name) -> str` — lowercase, spaces→underscores, strip specials

Intern vs fulltime branching is fully implemented.

dashboard.py

Entry point: `streamlit run dashboard.py`

Routing: show_onboarding → profile selector → full dashboard

Full dashboard has:

- Sidebar: avatar, switch profile, DB stats, last run time, "Run job search" button
- Tab 1 Jobs: filterable table with score badges, status dropdown (persists immediately), eye toggle to expand job detail panel with dimension bar chart + reasons/flags/ATS misses
- Tab 2 Analytics: score histogram, jobs by source, by status, top companies, ATS vs fit scatter
- Tab 3 Profile: read-only config view
- Tab 4 Settings: min score slider, weights sliders (must sum to 1.0), save to config.yaml, re-score button (double confirm), clear all jobs (double confirm), manage companies section

`_run_pipeline(config, slug)` handles the full scrape + score flow inside Streamlit, catches `SystemExit` from RPD limiter so Streamlit server doesn't die.

main.py CLI flags

```
python main.py                  # full run: scrape → score → display
python main.py --scrape-only
python main.py --score-only
python main.py --show
python main.py --rescore
python main.py --yes
python main.py --min-score 75
python main.py --profile manav
```

requirements.txt (current)

```
pyyaml>=6.0
python-dotenv>=1.0.0
httpx>=0.27.0
beautifulsoup4>=4.12.0
PyPDF2>=3.0.0
groq>=0.9.0
streamlit>=1.28.0
pydantic>=2.7.0
instructor>=1.3.0
pandas  # used by dashboard.py
```

Environment variables

```
GROQ_API_KEY=         # active default provider
ANTHROPIC_API_KEY=    # if provider: anthropic
GEMINI_API_KEY=       # if provider: gemini
OPENAI_API_KEY=       # if provider: openai
THEIRSTACK_API_KEY=   # optional — enables dynamic company discovery
```

Known issues / things to watch

- `config.py` still uses PyPDF2 — should migrate to pdfplumber for better text extraction
- HN sources key in config is `"hn"` not `"hackernews"` — dashboard checks `sources.get("hn")`

- `score_all_jobs()` uses `sys.exit(0)` for RPD limit — dashboard catches SystemExit, but main.py does not — should be changed to raise a custom exception instead
- `config.py` and `db.py` both have `load_config()` — db.py re-exports from config.py; callers should import from `db` for consistency

---

Next: Step 9 — Loguru + Pipeline Run Logging

Replace all `print()` statements with structured loguru logging. Add a `scrape_runs` table to track every pipeline execution end-to-end.

Goals:

1. Every module logs with loguru instead of print — consistent format, log levels, timestamps
2. `scrape_runs` table records: run_id, profile, started_at, finished_at, source, jobs_scraped, jobs_filtered, jobs_saved, jobs_scored, errors_json, avg_fit_score
3. Dashboard Tab 2 Analytics gets a "Run history" section showing recent pipeline runs
4. Log file written to `logs/{profile}/agent.log` with rotation (10MB, 7 days)
5. `sys.exit(0)` in scorer replaced with a custom `RateLimitReached` exception — caught in main.py and dashboard.py, logged properly instead of crashing

Full spec coming when ready to build.